Week 5 – Security Testing

Security Testing Summary

1. Introduction

Security testing was conducted to evaluate the robustness and reliability of the web application against potential security threats and vulnerabilities. The primary objective of this phase was to identify weaknesses that could compromise the confidentiality, integrity, and availability of the system.

The testing process focused on identifying issues related to improper input validation, cross-site scripting (XSS), session management, authentication mechanisms, and missing security configurations. Both automated and manual testing approaches were utilized to ensure comprehensive coverage of potential security risks.

Automated vulnerability scanning was performed using **OWASP ZAP**, while manual testing was conducted on the **SauceDemo** application.

2. Testing Approach

The security testing strategy consisted of two main components:

2.1 Automated Security Testing

Automated testing was conducted using OWASP ZAP to identify common web application vulnerabilities.

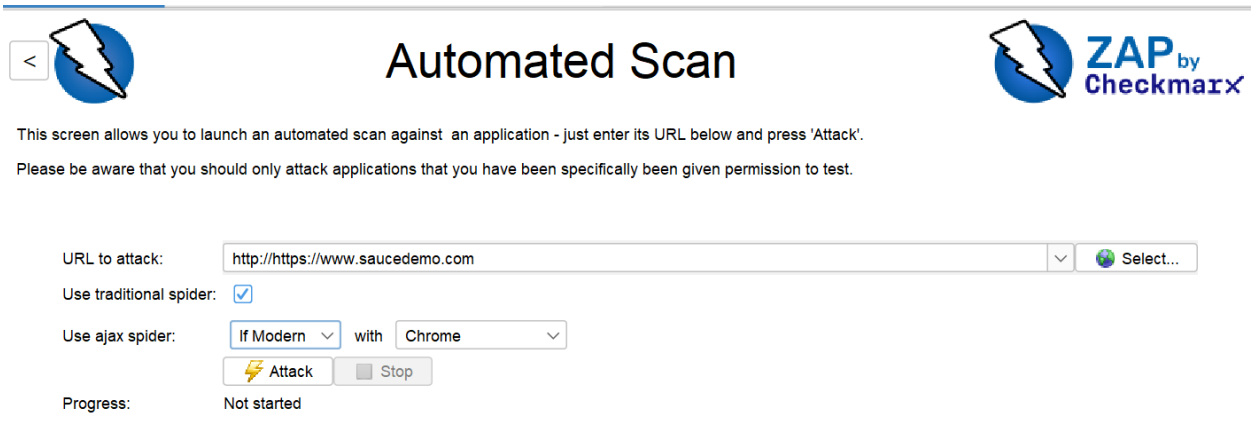

Figure 1: OWASP ZAP Automated Scan Interface

The tool performed spider scanning and active scanning to detect issues such as:

- Missing HTTP security headers

- Cross-domain misconfigurations
- Information disclosure vulnerabilities
- Weak security configurations

The automated scan provided a quick overview of security weaknesses present in the application.

2.2 Manual Security Testing

Manual testing was performed to validate application behavior against real-world attack scenarios. The following aspects were tested:

- Input validation mechanisms
- Cross-Site Scripting (XSS) protection
- Login and logout functionality
- Session timeout and session management

Manual testing ensured that the application was not only secure in configuration but also resistant to practical attack techniques.

3. Summary of Vulnerabilities Identified

The automated scan detected a total of **11 alerts**, categorized into different severity levels:

- **Medium Severity Issues:** Related to missing critical security headers
- **Low Severity Issues:** Related to configuration weaknesses
- **Informational Alerts:** General observations and best practice recommendations

Key Vulnerabilities Identified

1. Content Security Policy (CSP) Header Not Set
2. Cross-Domain Misconfiguration
3. Missing Anti-Clickjacking Header
4. Subresource Integrity Attribute Missing
5. Strict-Transport-Security Header Not Set
6. X-Content-Type-Options Header Missing
7. Information Disclosure – Suspicious Comments

Figure 2: OWASP ZAP Alerts Panel Showing Detected Vulnerabilities

Most of these vulnerabilities were related to **missing HTTP security headers**, which are essential for protecting web applications against modern security threats.

4. Manual Testing Findings

Manual testing revealed the following observations:

4.1 Input Validation

The application successfully handled malicious inputs such as script tags and SQL-like queries. Invalid inputs were rejected, and appropriate error messages were displayed.

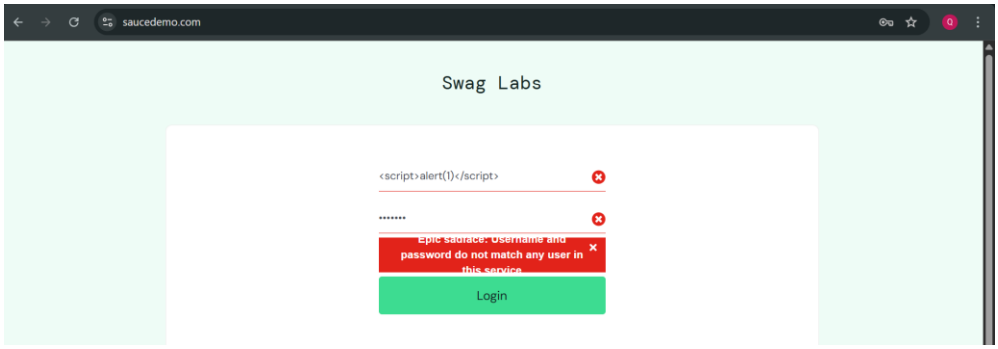

Figure 3: Input Validation Test (Invalid Script Input)

4.2 Cross-Site Scripting (XSS)

Basic XSS payloads were tested on input fields. The application did not execute injected scripts, indicating protection against simple XSS attacks.

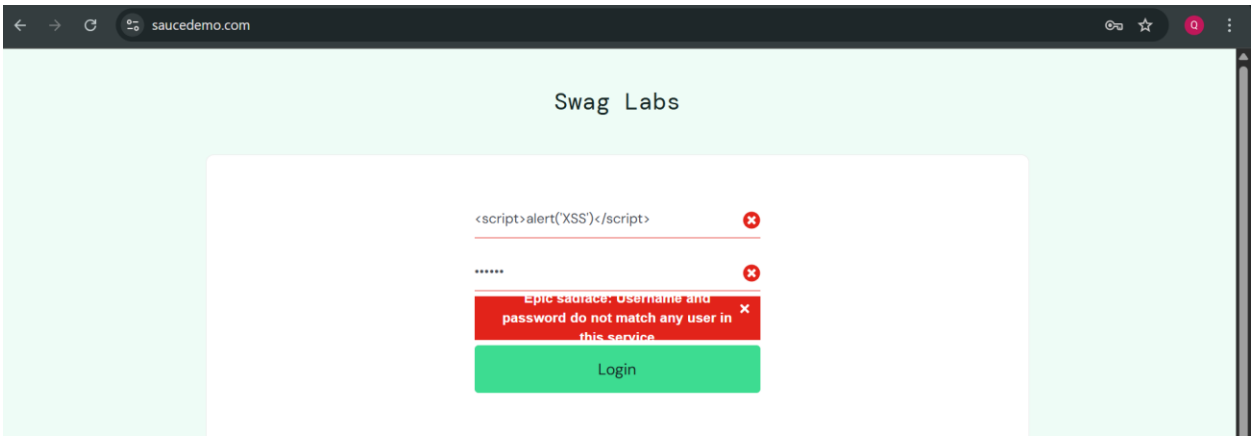

Figure 4: Cross-Site Scripting (XSS)

4.3 Authentication (Login/Logout)

The login functionality worked correctly with valid credentials. After logout, users were not able to access restricted pages using the browser back button, ensuring proper session termination.

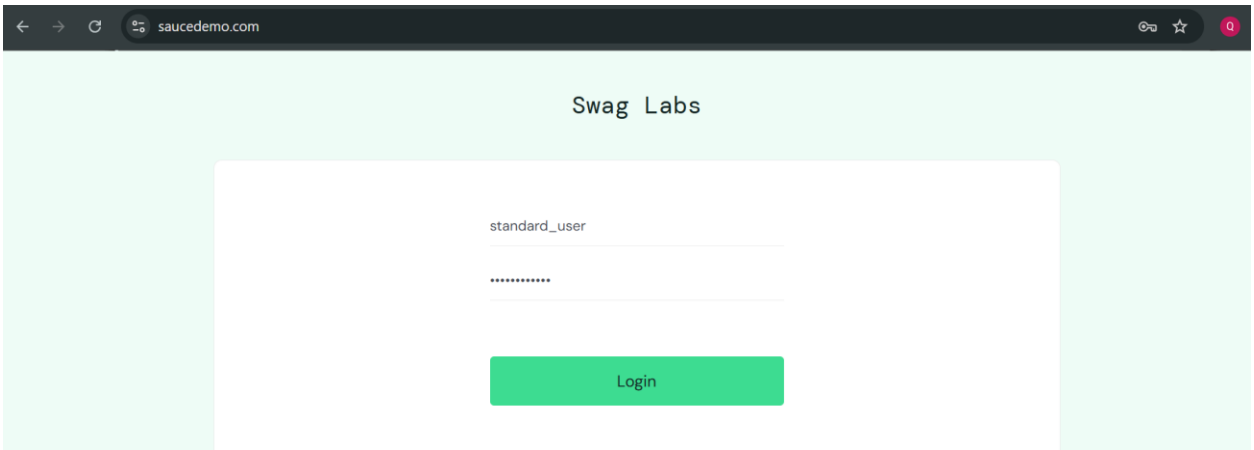

Figure 5: Authentication Login

4.4 Session Management

The application session remained active after a period of inactivity. This indicates a lack of strict session timeout policies, which could pose a security risk in real-world scenarios.

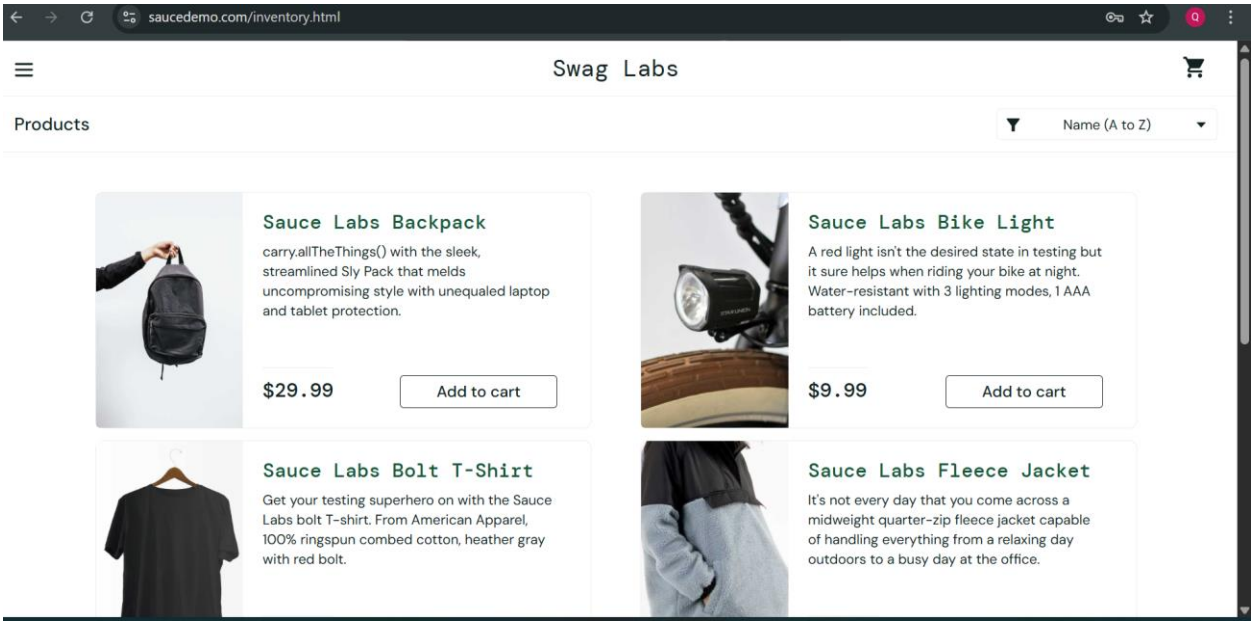

Figure 6: Session Management

5. Risk Analysis

The overall risk level of the application is considered **Low to Medium** based on the findings.

- No **High or Critical vulnerabilities** were detected
- Most issues were related to **configuration and security headers**
- Manual testing confirmed that **core functionalities are secure**

However, the absence of proper security headers and session timeout mechanisms could expose the application to potential risks if deployed in a production environment.

6. Recommendations

To improve the security posture of the application, the following recommendations are suggested:

- Implement **Content Security Policy (CSP)** to prevent XSS attacks
- Add **X-Frame-Options header** to protect against clickjacking
- Enable **Strict-Transport-Security (HSTS)** to enforce secure communication
- Include **X-Content-Type-Options header** to prevent MIME-type sniffing
- Apply **Subresource Integrity (SRI)** for external resources

- Configure proper **CORS policies** to avoid cross-domain vulnerabilities
- Implement **automatic session timeout** for inactive users
- Remove unnecessary comments from source code to prevent information leakage

7. Conclusion

The security testing phase provided valuable insights into the vulnerabilities and security posture of the application. The use of automated tools such as OWASP ZAP, combined with manual testing techniques, ensured a comprehensive evaluation of the system.

Although no critical vulnerabilities were identified, several improvements are required to enhance the overall security of the application. By implementing the recommended fixes, the application can significantly reduce its exposure to common web security threats and improve its resilience against potential attacks.